



C# 아키텍트의 설계도: 속성, 메모리, 최적화

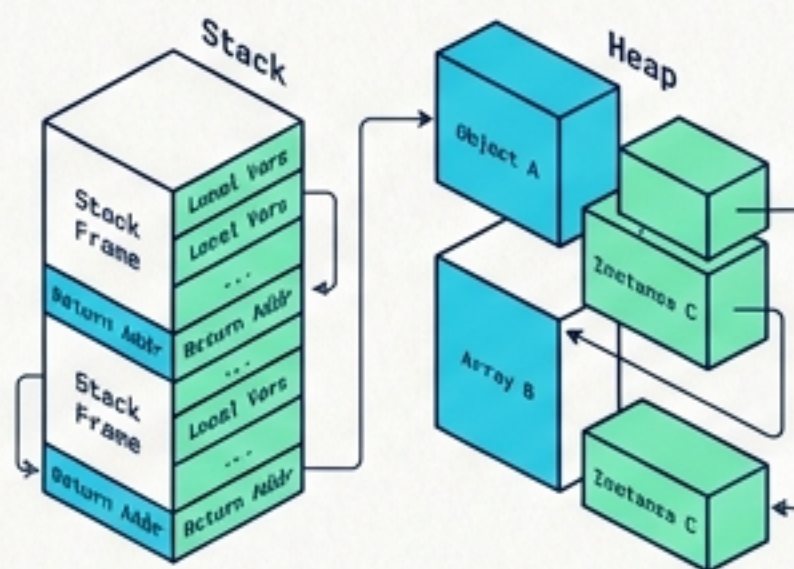
견고하고 효율적인 객체지향 코드를 구축하는 3가지 핵심 원리

설계의 3대 기둥



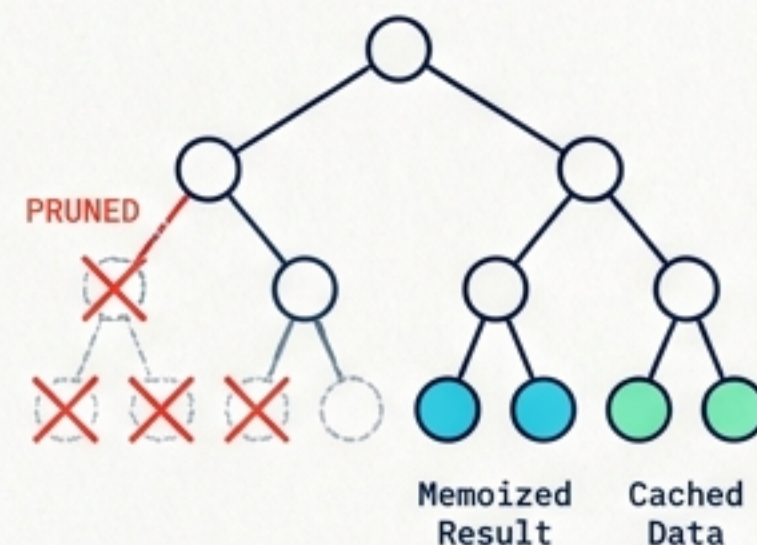
데이터 무결성

속성(Properties)과 캡슐화를
통한 안전한 데이터 통제



메모리 아키텍처

값(Value)과 참조(Reference)의
근본적인 차이 이해



연산 최적화

재귀(Recursion)의 함정과
메모화(Memoization)를 통한
성능 혁명

[문제 제기] 개방된 데이터의 치명적 결함

```
public int width;  
public int height;
```

누구나 접근 가능한 Public 필드

```
box.width = -10; 
```

증상: 사각형의 넓이가 음수(-)로 계산되는 논리적 오류 발생.

원인: 외부에서 아무런 검증 없이 클래스 내부 변수에 직접 접근하고 수정할 수 있기 때문.

[해결책] 검문소 설치: Getter와 Setter

Stage 1
입력(Input)

width = -10

Stage 2
검증소(Setter Method)

```
if (width > 0) {  
    this.width = width;  
}
```

Stage 3
금고(Private Field)

private int width;

접근 제한자를 private으로 닫고, 오직 메서드를 통해서만 값을 통제합니다.
(단점: 코드가 길어짐)

[진화] C#의 우아한 설계: 속성(Properties)

Code Anatomy

속성 이름 (대문자 시작)

```
public int Width  
{
```

```
    get { return width; }
```

```
    set { if (value > 0) width = value; }
```

```
}
```

C#의 숨겨진 키워드 (입력된 값)

값을 읽을 때 자동 호출

값을 할당할 때 자동 호출

인사이트: 메서드처럼 안전하고, 변수처럼 직관적이다.
`box.Width = 10;` 처럼 변수 다루듯 코딩하지만, 내부적으로는 `set` 블록이 실행되어 완벽한 통제력을 유지합니다.

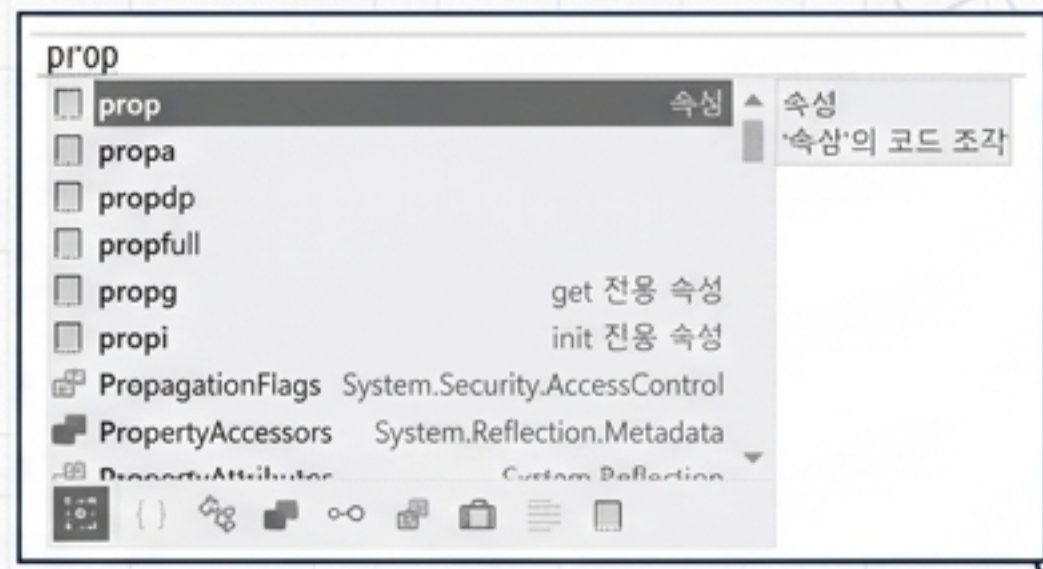
[생산성] 아키텍트의 도구: 자동 구현과 스니펫



```
Box box = new Box(-10, -20);
box.
```

- Area
- Equals
- GetHashCode
- GetType
- Height** int Box.Height { get; set; }
- ToString
- Width
-

IDE에서 이 아이콘(스패너)을 본다면,
그것은 변수가 아니라 '속성'입니다.



입력: prop + [Tab] [Tab]

```
public int MyProperty { get; set; }
```

자동 구현 속성. 검증 로직이 필요
없을 때 코드를 극도로 압축합니다.

입력: propfull + [Tab] [Tab]

```
private int myVar;

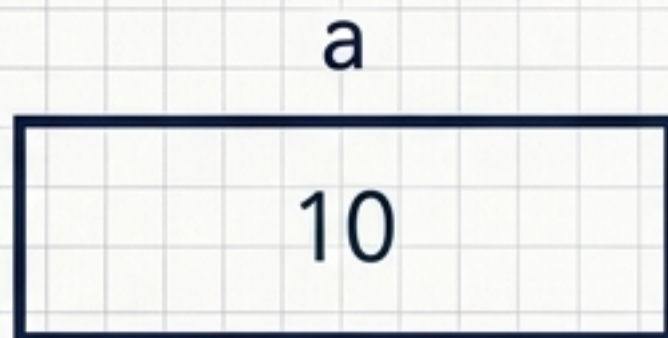
public int MyProperty
{
    get { return myVar; }
    set { myVar = value; }
}
```

전체 속성 뼈대 자동 생성.

데이터의 두 가지 체급: 값(Value) vs 참조(Reference)

[Panel 1: 값 (기본 자료형)]

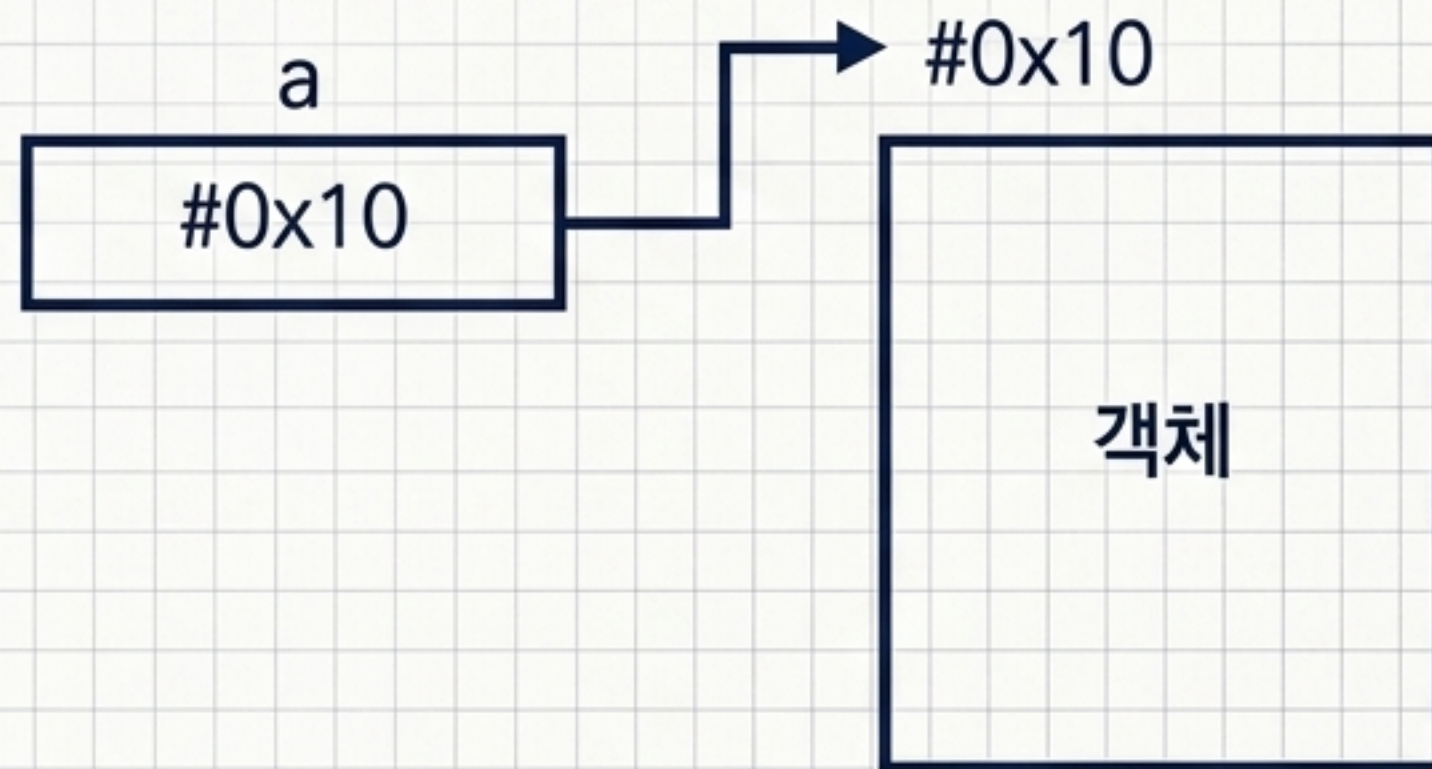
```
int a = 10;
```



특징: 가볍고 빠름. 메모리(Stack) 상자에
데이터가 직접 들어감.

[Panel 2: 참조 (객체 자료형)]

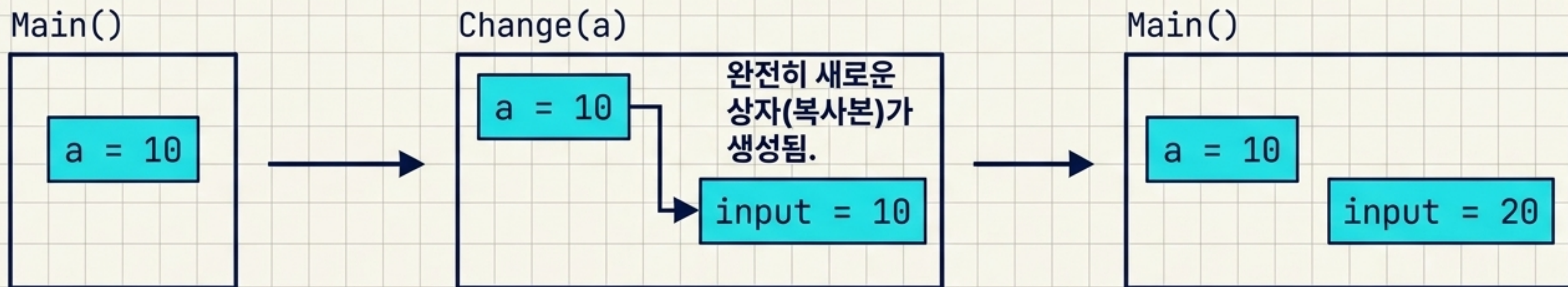
```
Random a = new Random();
```



특징: 무겁고 복잡함. 변수에는 주소(위치표)만
주고 실제 데이터는 힙(Heap)에 보관.

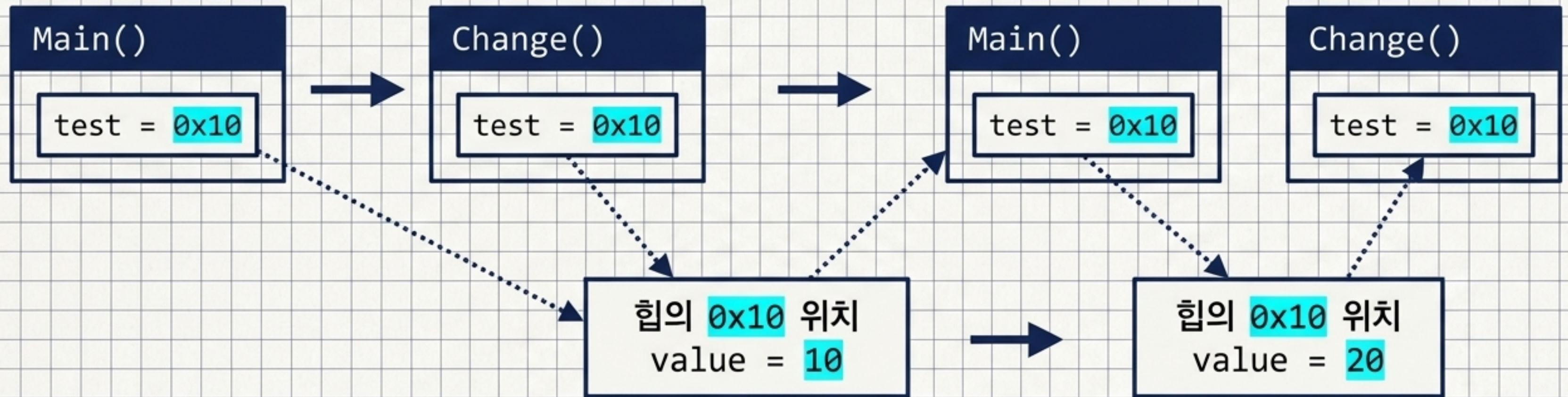
[동작 원리]

독립적인 세계: 값 복사 (Pass by Value)



**결과: 원본과 복사본은 서로에게 어떤
영향도 주지 않는 완벽한 독립 상태.**

[동작 원리] 연결된 운명: 참조 복사 (Pass by Reference)



결과: 데이터가 아닌 '주소'가 복사됨.
동일한 객체를 공유하므로, 메서드 내부에서 수정하면 원본도 변경됨!

[종합 진단] 값 타입 vs 참조 타입 퀵 레퍼런스

비교 기준	값 타입 (Value Type)	참조 타입 (Reference Type)
대상 데이터	int, float, bool 등 기본형	class, 배열 등 복합 객체
저장 위치	스택 (Stack) - 데이터 직접 저장	스택(주소) + 힙(실제 데이터)
메서드 전달	복사본 생성 (독립적)	주소 전달 (원본 공유)
원본 변경 여부	안전함 (변경 안 됨)	주의 필요 (변경됨)



Architect's Note: 객체(참조)를 메서드에 넘길 때는 원본이 수정될 수 있음을 항상 설계에 반영해야 합니다.

[응용] 재귀 메서드: 자기 자신을 부르는 우아함

$$F_n := \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n > 1. \end{cases}$$

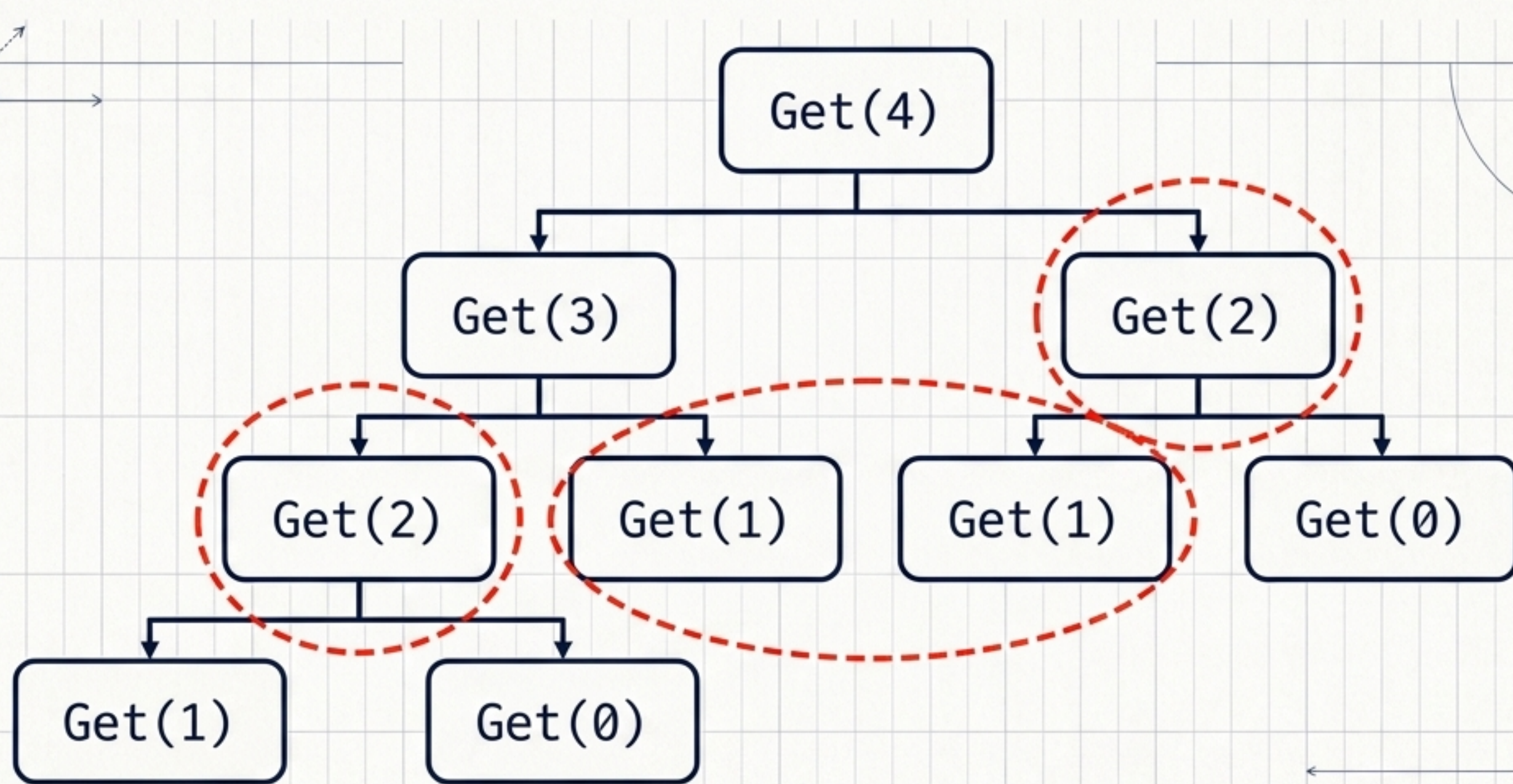
수학의 점화식

```
public long Get(int i)
{
    if (i <= 0) { return 0; }
    if (i == 1) { return 1; }
    return Get(i - 2) + Get(i - 1);
}
```

종료 조건 (무한 루프를
막는 필수 안전장치)

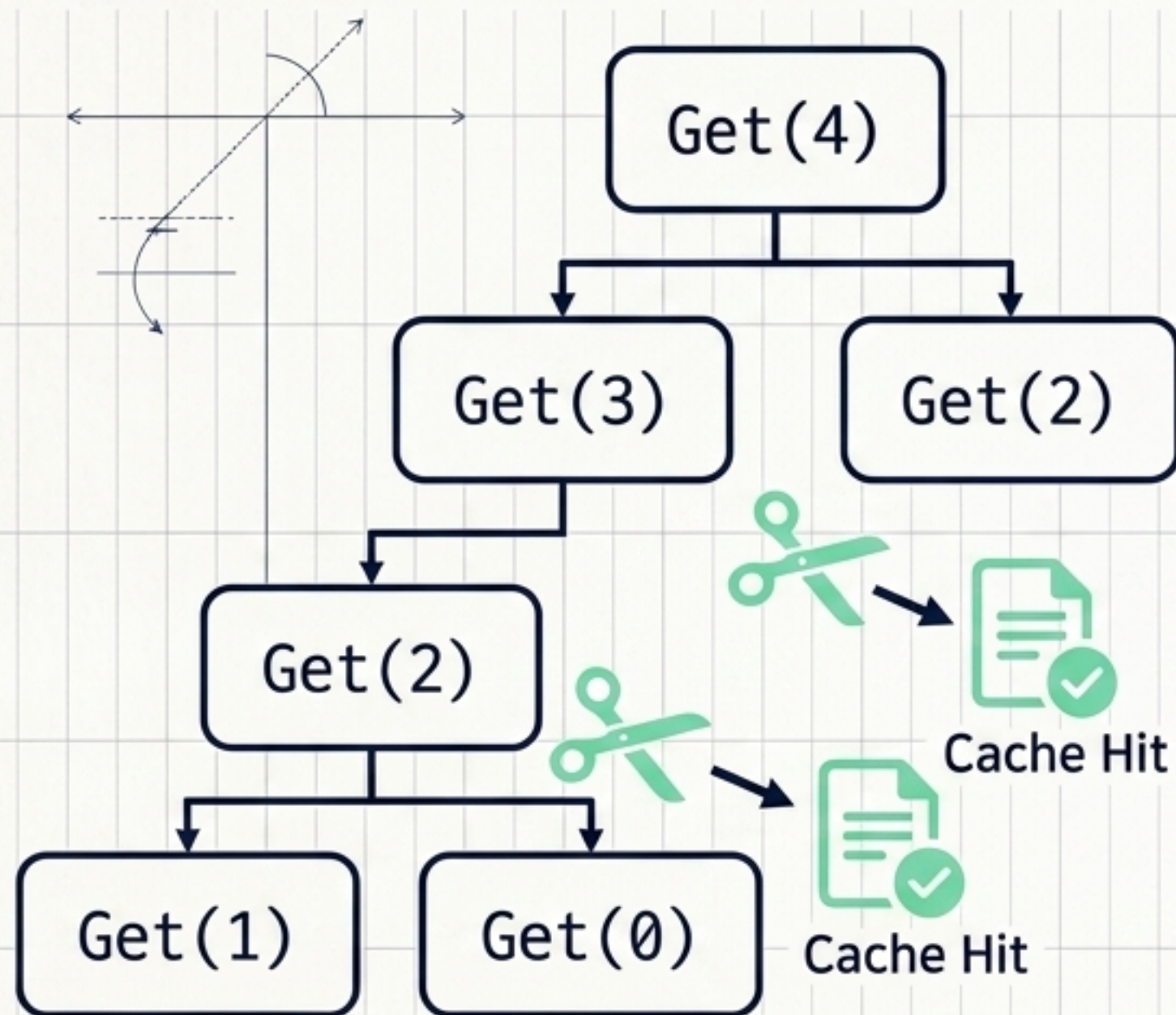
return Get(i - 2) + Get(i - 1); 자기 자신 호출

[한계] 단순 재귀의 숨겨진 비용 (The Hidden Cost)



숫자가 커질수록 (예: `Get(40)`) 동일한 계산을 수백만 번 반복하게 되어 프로그램이 얼어붙는 성능 붕괴(Performance Collapse)가 발생합니다.

[최적화] 성능 혁명: 메모화 (Memoization)



```
private static Dictionary<int, long> memo =  
    new Dictionary<int, long>();
```

```
public static long Get(int i)  
{
```

```
    if (i < 0) { return 0; }
```

```
    if (i == 1) { return 1; }
```

```
    if (memo.ContainsKey(i)) return memo[i];
```

```
    long value = Get(i - 2) + Get(i - 1);
```

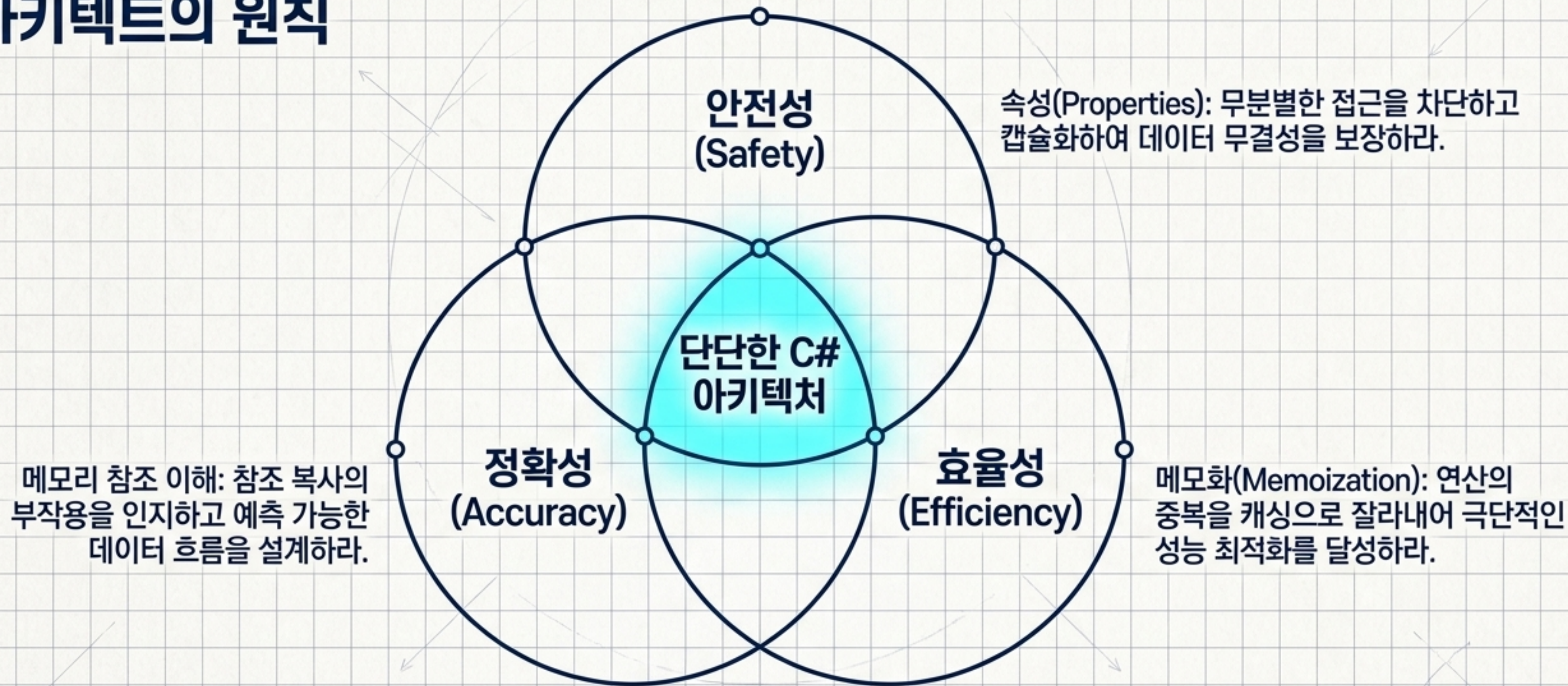
```
    memo[i] = value;
```

```
    return value;
```

```
}
```

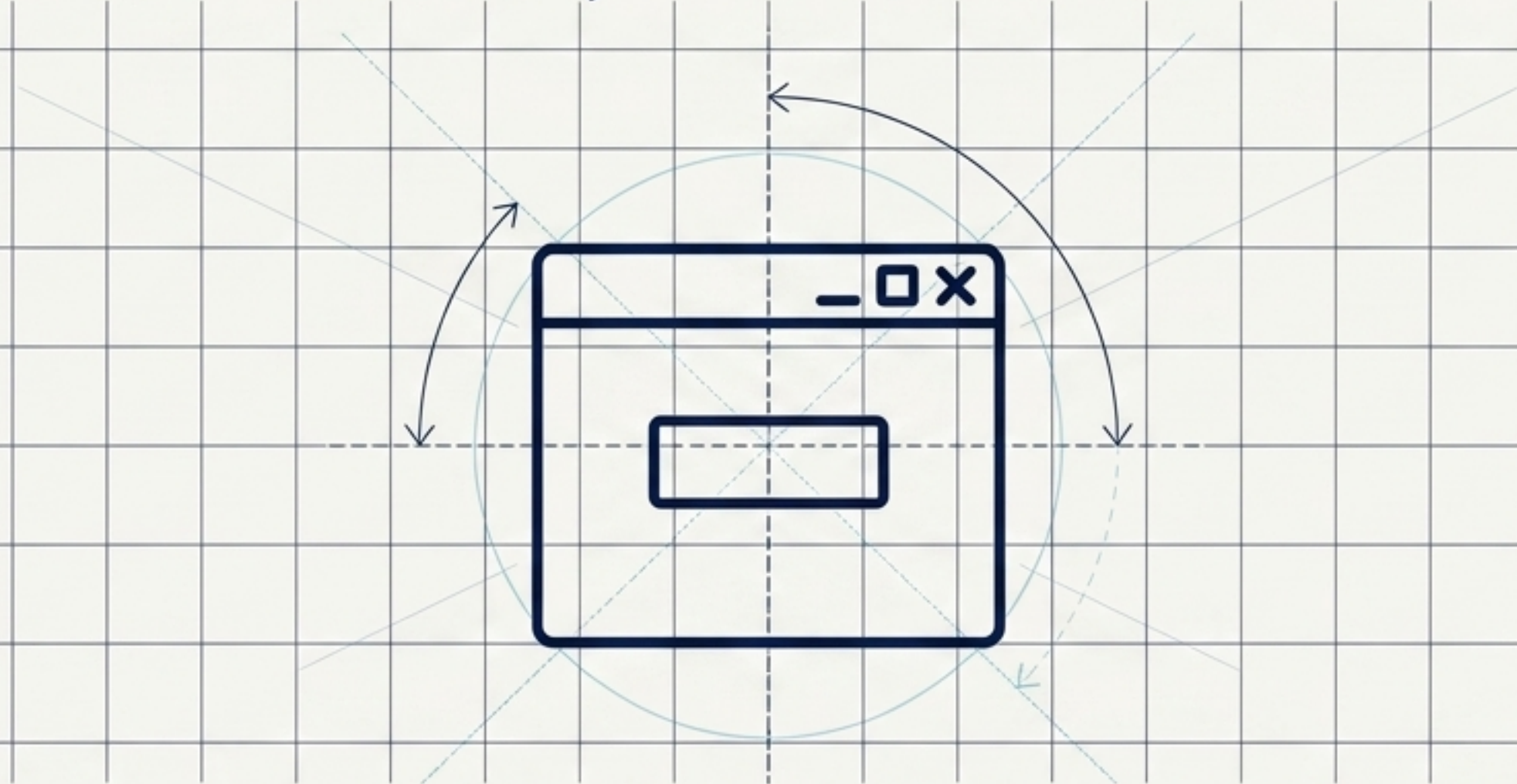
한 번 계산한 값은
수첩(Dictionary)에 적어두고,
다시 묻지 않는다.

[핵심 종합] 단단한 C# 코드를 위한 아키텍트의 원칙



이 세 가지를 완벽히 통제하는 자가 진정한 C# 마스터로 거듭납니다.

아키텍처 완성, 그리고 다음 단계로.



데이터를 안전하게 감싸고, 메모리의 동작을 꿰뚫어 보며, 최적화된 로직을 짜는 방법을 익혔습니다.

이제 이 강력한 무기들을 눈에 보이는 화면(GUI)에 연결할 차례입니다.

다음 장에서는 오늘 배운 개념들이 윈도우 폼(Windows Forms)에서 어떻게 사용자와 상호작용하는지 살펴봅니다.